

O1. Query Processing Pipeline

```
ALGORITHM QueryProcessingPipeline(SQL_Query)
BEGIN
    TokenList <- Lexer(SQL_Query)
    ParseTree <- Parser(TokenList)
    IF ParseTree is INVALID THEN
        RETURN Error("Syntax Error")

    IF NOT ValidateSchema(ParseTree) THEN
        RETURN Error("Semantic Error")

    RelAlgebraExpr <- ConvertToRelationalAlgebra(ParseTree)

    CandidatePlans <- GenerateEquivalentPlans(RelAlgebraExpr)
    FOR EACH plan IN CandidatePlans DO
        plan.cost <- EstimateCost(plan)
    END FOR
    BestPlan <- SelectMinimumCostPlan(CandidatePlans)

    ExecutablePlan <- GenerateExecutionCode(BestPlan)
    Result <- Execute(ExecutablePlan)

    RETURN FormatOutput(Result)
END
```

O2. Cost-Based Query Optimization (Join Ordering)

```
ALGORITHM CostBasedJoinOrdering(Relations[R1..Rn])
BEGIN
    FOR EACH subset S OF Relations (size increasing) DO
        IF |S| = 1 THEN
            OptCost[S] <- AccessCost(S)
            OptPlan[S] <- ScanPlan(S)
        ELSE
            best <- INFINITY
            FOR EACH split S INTO (S1, S2) DO
                cost <- OptCost[S1] + OptCost[S2] + JoinCost(S1,S2)
                IF cost < best THEN
                    best <- cost; bestSplit <- (S1,S2)
            END FOR
            OptCost[S] <- best
            OptPlan[S] <- JoinPlan(OptPlan[S1], OptPlan[S2])
        END IF
    END FOR
    RETURN OptPlan[Relations], OptCost[Relations]
END

FUNCTION JoinCost(S1, S2)
BEGIN
    outerPages <- Pages(S1); innerPages <- Pages(S2)
    RETURN outerPages + CEIL(outerPages/(B-2)) * innerPages
END

FUNCTION AccessCost(R)
BEGIN
    IF IndexExists(R) THEN RETURN IndexScanCost(R)
    ELSE RETURN Pages(R)
END
```

O3. Nested Loop Join (with Complexity)

```
ALGORITHM NestedLoopJoin(R, S, condition C, BufferPages B)
BEGIN
    Result <- EMPTY SET
    FOR EACH block Br OF R (B-2 buffer pages) DO
        LOAD Br
        FOR EACH block Bs OF S DO
            LOAD Bs
            FOR EACH tuple r IN Br DO
                FOR EACH tuple s IN Bs DO
                    IF C(r,s) THEN
                        Result <- Result UNION {r,s}
                END FOR
            END FOR
        END FOR
    END FOR
    RETURN Result
END
```

```

                IF r SATISFIES C WITH s THEN
                    Result <- Result UNION Combine(r,s)
            RETURN Result
END

// Complexity: Let Pr,Ps = pages of R,S ; Nr,Ns = tuples
// Simple NLJ   : I/O = Pr + (Nr * Ps)           -> O(Nr*Ns) comparisons
// Block NLJ    : I/O = Pr + CEIL(Pr/(B-2))*Ps -> O(Pr*Ps/B)

```

O4. Deadlock Detection (Wait-For Graph)

```

ALGORITHM DetectDeadlock(TransactionSet T)
BEGIN
    WFG <- EMPTY GRAPH
    FOR EACH Ti IN T DO
        FOR EACH (Ti waits for Tj) DO
            AddEdge(WFG, Ti, Tj)

    visited <- {}; inStack <- {}
    FOR EACH node N IN WFG DO
        IF N NOT IN visited THEN
            IF HasCycledDFS(N, visited, inStack, WFG) THEN
                RETURN "Deadlock Detected"
    RETURN "No Deadlock"
END

FUNCTION HasCycledDFS(node, visited, inStack, WFG)
BEGIN
    visited <- visited U {node}
    inStack <- inStack U {node}
    FOR EACH nb IN AdjacentNodes(WFG, node) DO
        IF nb IN inStack THEN RETURN TRUE
        ELSE IF nb NOT IN visited THEN
            IF HasCycledDFS(nb, visited, inStack, WFG) THEN
                RETURN TRUE
    inStack <- inStack - {node}
    RETURN FALSE
END

```

O5. Basic Two-Phase Locking (2PL)

```

ALGORITHM TwoPhaseLocking(Transaction T)
BEGIN
    Phase <- "GROWING"; LockSet[T] <- {}

    FOR EACH data item D needed by T DO
        IF Phase = "SHRINKING" THEN
            RETURN Error("New lock after release not allowed")
        RequestLock(T, D, mode)
        WHILE Conflicting(D, mode) DO
            WAIT
        GrantLock(T, D, mode)
        LockSet[T] <- LockSet[T] U {D}
        Execute(T, D)

    Phase <- "SHRINKING"
    FOR EACH D IN LockSet[T] DO
        ReleaseLock(T, D)
    Commit(T)
END

```

O6. Timestamp Ordering - Thomas' Write Rule

```

ALGORITHM ThomasWriteRule(Ti, D, Op)
BEGIN
    IF Op = "READ" THEN
        IF TS(Ti) < D.Write_TS THEN Abort(Ti)
    ELSE
        ReadValue(D)

```

```

        D.Read_TS <- MAX(D.Read_TS, TS(Ti))

    ELSE IF Op = "WRITE" THEN
        IF TS(Ti) < D.Read_TS THEN
            Abort(Ti)
        ELSE IF TS(Ti) < D.Write_TS THEN
            IGNORE the write          // Thomas' Write Rule
        ELSE
            WriteValue(D)
            D.Write_TS <- TS(Ti)
    END

```

O7. ARIES Recovery (Analysis. Redo. Undo)

```

ALGORITHM ARIES_Recovery(LogFile, LastCheckpoint)
BEGIN
    // ANALYSIS
    DirtyPageTable <- Load(LastCheckpoint)
    TransTable <- Load(LastCheckpoint)
    FOR EACH log record L from checkpoint to end DO
        IF L.type="UPDATE" THEN
            IF L.pageID NOT IN DirtyPageTable THEN
                DirtyPageTable[L.pageID] <- L.LSN
                TransTable[L.transID].lastLSN <- L.LSN
            ELSE IF L.type="COMMIT" THEN
                TransTable[L.transID].status <- "COMMITTED"
            ELSE IF L.type="END" THEN
                REMOVE L.transID FROM TransTable
    Losers <- {T IN TransTable | status != COMMITTED}

    // REDO
    RedoLSN <- MIN(recLSN in DirtyPageTable)
    FOR EACH log record L from RedoLSN to end DO
        IF L.type="UPDATE" AND L.pageID IN DirtyPageTable THEN
            IF L.LSN >= DirtyPageTable[L.pageID]
                AND PageLSN(L.pageID) < L.LSN THEN
                RedoAction(L)
                PageLSN(L.pageID) <- L.LSN

    // UNDO
    ToUndo <- {T.lastLSN FOR T IN Losers}
    WHILE ToUndo NOT EMPTY DO
        L <- record with MAX(LSN) IN ToUndo
        REMOVE L FROM ToUndo
        IF L.type="UPDATE" THEN
            UndoAction(L); WriteCLR(L, L.prevLSN)
            IF L.prevLSN EXISTS THEN ADD L.prevLSN TO ToUndo
            ELSE WriteLog("END", L.transID)
        ELSE IF L.type="CLR" THEN
            IF L.undoNextLSN EXISTS THEN ADD L.undoNextLSN TO ToUndo
            ELSE WriteLog("END", L.transID)
    END

```

O8. Shadow Paging Recovery

```

ALGORITHM ShadowPaging(Transaction T)
BEGIN
    ShadowPT <- COPY of on-disk page table
    CurrentPT <- COPY of ShadowPT

    FOR EACH write(P, newValue) by T DO
        newFrame <- AllocateFreeDiskFrame()
        WRITE newValue INTO newFrame
        CurrentPT[P] <- newFrame

    IF T COMMITS THEN
        FlushAllPages(CurrentPT)
        WriteAtomically(PageTablePointer, CurrentPT)
        ShadowPT <- CurrentPT

```

```

        DiscardOldFrames(old ShadowPT - CurrentPT)
    ELSE IF T ABORTS / CRASH THEN
        DiscardFrames(CurrentPT - ShadowPT)
        CurrentPT <- ShadowPT
END

```

O9. Immediate Update Recovery (UNDO/REDO)

```

ALGORITHM ImmediateUpdateRecovery(LogFile)
BEGIN
    UndoList <- []; RedoList <- []

    FOR EACH log record L IN LogFile (forward) DO
        IF L.type="START" THEN ADD L.transID TO UndoList
        ELSE IF L.type="COMMIT" THEN
            REMOVE L.transID FROM UndoList
            ADD L.transID TO RedoList

    FOR EACH log record L IN LogFile (BACKWARD) DO
        IF L.type="UPDATE" AND L.transID IN UndoList THEN
            WriteToDisk(L.dataItem, L.oldValue)

    FOR EACH log record L IN LogFile (FORWARD) DO
        IF L.type="UPDATE" AND L.transID IN RedoList THEN
            WriteToDisk(L.dataItem, L.newValue)
END

```

O10. DB Connectivity Manager (Pooling. Execution. Rollback)

```

ALGORITHM DBConnectionManager(MaxPoolSize)
BEGIN
    ConnectionPool <- EMPTY QUEUE; ActiveConnections <- 0

    FUNCTION GetConnection()
    BEGIN
        IF ConnectionPool NOT EMPTY THEN
            conn <- DEQUEUE(ConnectionPool)
        ELSE IF ActiveConnections < MaxPoolSize THEN
            conn <- CreateNewConnection()
            ActiveConnections <- ActiveConnections + 1
        ELSE
            WAIT until connection is released
            conn.status <- "IN_USE"
        RETURN conn
    END

    FUNCTION ExecuteQuery(conn, SQL)
    BEGIN
        TRY
            conn.BeginTransaction()
            result <- conn.Execute(SQL)
            conn.Commit()
            RETURN result
        CATCH Exception e
            conn.Rollback()
            LogError(e); RAISE e
        FINALLY
            ReleaseConnection(conn)
    END

    FUNCTION ReleaseConnection(conn)
    BEGIN
        IF conn.isValid() THEN
            conn.status <- "IDLE"
            ENQUEUE(ConnectionPool, conn)
        ELSE
            CLOSE(conn)
            ActiveConnections <- ActiveConnections - 1
    END

```

END